

Frontend technikai interjú felkészülés

React, Redux, Next.js, migration, performance, tooling, AI-assisted workflow

Mobilról olvasható, interjú-orientált jegyzet. A cél nem definíciók bemagolása, hanem hogy 1-2 percben szakmailag tisztán el tudd mondani a lényegét, és legyen kész válaszod a tipikus kérdésekre.

Használat: először olvasd át a "Top kérdések" részt, utána a részletes fejezeteket. Interjú előtt elég a kiemelt angol válaszokat átfutni.

Készült: az Artec 3D Senior Front-End Developer körre, a hirdetés, a CV-d és az első technikai beszélgetés alapján.

0. Gyors tartalom

- Pozicionálás és bemutatkozás
- Projektösszefoglaló: Docler, belső toolok, modular React app
- React működés: render, re-render, virtual DOM, reconciliation, hydration
- Hooks, memoization, performance, nagy listák
- State management: props, Context API, Redux, Redux-Saga, React Query
- Next.js, SSR, React 19 - csak amennyit interjúra tudni kell
- Reusable components, Figma, UI quality, CSS és JS alapok
- Legacy migration: jQuery/Rails vagy régi React irányból modern Reactbe
- Code quality toolok, tesztek, visual regression, CI
- REST, GraphQL, API collaboration
- AI-assisted development: hogyan add el profin
- 30+ valószínű interjúkérdés mintaválasszal

1. Pozicionálás: hogyan add el magad

A fő üzenet: senior frontend engineer vagy React/TypeScript fókusszal, de nem csak képernyőket raksz össze. Értesz komplex frontend architektúrához, API integrációhoz, release-hez, mentoringhoz és automation/toolinghoz is. A full-stack tapasztalatot pluszként add el, de a fókusz maradjon frontend.

30 másodperces bemutatkozás

Angolul:

I have been working as a software engineer for around 16 years, with a BSc degree in Software Engineering. My strongest recent experience is frontend development with React and TypeScript, especially complex production applications with reusable UI patterns, API integrations, feature flags, localization, and release coordination. I also have full-stack and tooling experience with Node.js, which helps me collaborate effectively with backend teams and understand the wider system around the frontend.

2 perces bemutatkozás

Angolul:

I have been working as a software engineer for around 16 years, and I have a BSc degree in Software Engineering. Over the years I have worked in different industries, including iGaming, e-learning, mobile game development, and most recently streaming and high-traffic web applications.

I spent the last 8 years at Docler Holding in Luxembourg, where I changed teams and responsibilities several times. I started in a frontend team working with Haxe, then moved for a period into a more full-stack role, working with Node.js and React. During that full-stack period, the frontend work was mostly internal tools, operational UIs, and test client applications.

After that, I moved back to frontend development, where my main focus became React and TypeScript. In the last few years I worked mostly on a modular React application with Redux and Redux-Saga, REST and

GraphQL integrations, feature flags, localization, and reusable UI patterns. Besides hands-on coding, I also mentored developers, managed releases, and in some projects acted as a technical owner, coordinating business needs, technical discussions, and cross-team releases.

Mit ne hangsúlyozz túl

- Ne azt mondd, hogy full-stack role-t keresel. Mondd: frontend a fókusz, full-stack tapasztalat plusz.
- Ne állítsd, hogy a belső toolok public-facing production appok voltak. Mondd: internal tools, operational UIs, test clients.
- Ne használd túl erősen a “business logic was completely separated” mondatot. Inkább: “we tried to keep a clear separation”.
- A modular appot ne nevezd automatikusan microfrontendnek. Inkább: modular frontend architecture inside one main application.

2. Projektösszefoglaló: mit mondj az elmúlt évekről

Átfogó projekt-narratíva

A CV és a hirdetés alapján a legjobb történet: hosszú frontend múlt, high-traffic környezet, komplex modular React app, de közben volt backend/tooling tapasztalat is. Ez jól passzol a senior role-hoz, ha nem akarsz túlzottan Rails/jQuery specialistának tűnni.

Angolul:

In terms of projects, the company had several main client-side products, and I worked longer-term on two of them. My most recent and most relevant work was on JasminCam, a modular React and TypeScript client application using Redux and Redux-Saga. It was designed to support multiple product configurations from the same codebase, with feature flags, localization, REST and GraphQL integrations, and reusable UI patterns.

Before that, I also worked in a more full-stack capacity on internal company tools. These were mostly internal operational UIs and test clients rather than public-facing production products. They included tab-based views for statistics, service status, release-related actions, and

buttons triggering service calls such as starting or stopping servers or supporting release workflows.

JasminCam / modular React app

Angolul:

The most recent project was a modular React application using Redux and Redux-Saga. It had a core application layer that provided the main layout, shared state handling, configuration, and core features. Most non-core functionality was implemented as independent feature modules.

The modules were loosely coupled and reacted to application-level events and state changes rather than directly depending on each other. On the UI side, the core layout exposed predefined placeholders or extension points. Depending on the internal navigation state and application context, modules could render their own UI into those areas.

It was not traditional browser URL-based routing. It was more of an internal navigation and routing mechanism. We also tried to separate UI, state, and business workflows: reducers and selectors handled state and derived data, Redux-Saga handled API calls and side effects, while React components mostly focused on rendering and user interaction.

Was it microfrontend?

Rövid válasz:

I would not call it a full microfrontend architecture, because the modules were still part of one main application and shared the same runtime and deployment lifecycle. It had microfrontend-like characteristics in terms of modularity and independent feature ownership, but technically it was closer to a modular frontend architecture inside one main application.

3. Top interjúkérdések - rövid válaszok

Ezek azok, amiket a következő körben leginkább várnék. A válaszok direkt interjú-stílusúak: nem túl hosszúak, de szakmailag elég erősek.

Q: Can you briefly introduce yourself and your recent frontend experience?

Interjú válasz:

I have around 16 years of software engineering experience. My strongest recent focus is React and TypeScript frontend development in high-traffic web applications. I have worked with Redux, Redux-Saga, REST and GraphQL APIs, feature flags, localization, reusable UI patterns, and release coordination. I also have Node.js and internal tooling experience, which helps me understand the system beyond the frontend.

Mit figyelnek: Pozicionálás: senior frontend, de system-aware.

Q: Can you explain the modular React application you worked on?

Interjú válasz:

It was a modular React application with a core application layer and independent feature modules. The core provided layout, configuration, shared state and core features. Feature modules reacted to application-level events and state changes, and could render UI into predefined extension points based on internal navigation state. This allowed the same codebase to support multiple product configurations.

Mit figyelnek: Core, modules, loose coupling, extension points.

Q: Why did you use Redux?

Interjú válasz:

I would not use Redux by default for every React app. But it is useful when an application has complex shared state, multiple modules reacting to the same application-level events, and a need for predictable state updates. In our case, Redux helped separate UI from state and allowed modules to communicate through actions and state changes instead of direct dependencies.

Mit figyelnek: Ne "global state" legyen az egyetlen válasz.

Q: Could the app be built without Redux?

Interjú válasz:

For smaller apps, yes. Local state, Context API with useReducer, and React Query can cover many cases. But for a large modular app with cross-module events, predictable updates, selectors, and complex async

flows, Redux can still be a good fit.

Mit figyelnek: Mutasd, hogy nem Redux-vallásos vagy.

Q: What triggers a React re-render?

Interjú válasz:

A component re-renders when its own state changes, when a parent re-renders, when it receives new props, when a context value it consumes changes, or when an external store subscription changes. A re-render does not always mean the real DOM changes; React still compares the output and commits only necessary DOM updates.

Mit figyelnek: Különítsd el: component re-render vs real DOM update.

Q: How does React know what to update in the DOM?

Interjú válasz:

React schedules an update after a state, props, context, or external store change. It runs the component again, creates a new virtual representation of the UI, compares it with the previous one during reconciliation, and commits the required changes to the real DOM.

Mit figyelnek: Virtual DOM + reconciliation + commit.

Q: How would you render one million items?

Interjú válasz:

I would not render one million DOM nodes directly. I would use pagination, infinite scrolling, or list virtualization/windowing with a library such as react-window or react-virtualized, so only the visible rows are rendered. I would also use stable keys, memoized rows where useful, and backend pagination if the data comes from an API.

Mit figyelnek: A jó válasz: ne rendereld le egyszerre.

Q: How would you migrate a legacy jQuery/Rails UI to React?

Interjú válasz:

I would avoid a big-bang rewrite. I would identify isolated areas that can be replaced safely, introduce React incrementally, keep existing behavior

stable, add tests around critical flows, and migrate in small reviewable steps. The goal is to reduce risk while gradually moving new functionality into React.

Mit figyelnek: Pont passzol a hirdetéshez.

4. React működés: amit tisztán tudni kell

Hogyan indul egy React app?

Ezt akkor kérdezhetik, ha azt akarják látni, hogy érted-e a React alap működését, nem csak komponenseket írsz.

```
// Simplified React SPA entry point
import { createRoot } from "react-dom/client";
import { App } from "./App";

const rootElement = document.getElementById("root");
const root = createRoot(rootElement);
root.render(<App />);
```

Angol válasz:

A typical React SPA starts from an HTML file with a root DOM element. The JavaScript bundle loads, React creates a root with `createRoot`, and renders the component tree into that root. After that, React updates the UI based on state, props, context, or external store changes.

Re-render vs DOM update

Ez az egyik legfontosabb különbség. Reactban nem "DOM element renderelődik újra", hanem component functionök futnak újra. A real DOM csak akkor módosul, ha React a reconciliation után tényleges különbséget talál.

Rövid szabály

Parent re-render esetén a child componentek alapból újra lefuthatnak, de ez nem jelenti azt, hogy a böngésző DOM-ban is minden megváltozik.

Mi triggerel re-rendert?

- Saját state változása: `setState`, `useReducer` dispatch.
- Parent component re-renderje, ami miatt a child is újra lefuthat.
- Új props, vagy referenciában változó object/array/function prop.
- Context value változása, ha a component fogyasztja a contextet.
- External store/subscription változása, például Redux store update.
- Key változása: ilyenkor React új component instance-t is létrehozhat.

Angol válasz:

A React component re-renders when its state changes, when its parent re-renders, when it receives new props, when a context value it consumes changes, or when an external store subscription changes. The important point is that a re-render is not the same as a real DOM update.

Virtual DOM és reconciliation

A "virtual DOM" leegyszerűsítve React belső reprezentációja arról, hogy hogyan kellene kinéznie az UI-nak. State/props változás után React újra előállítja ezt a reprezentációt, összehasonlítja az előzővel, majd csak a szükséges változásokat commitolja a real DOM-ba. A folyamat neve reconciliation.

Angol válasz:

When something changes, React runs the relevant component functions again and creates a new virtual representation of the UI. During reconciliation it compares this new tree with the previous one, determines what changed, and then commits only the necessary updates to the real DOM.

Hydration

Hydration főleg SSR/Next.js környezetben releváns. A szerver előre HTML-t küld, a browser megjeleníti, majd a kliens oldali React betöltődik, és "rákapcsolódik" a meglévő HTML-re: event handlereket köt, state-et inicializál, interaktívvá teszi az oldalt.

Angol válasz:

Hydration is the process where React attaches client-side behavior to HTML that was already rendered on the server. The page can be visible before it is fully interactive, and React then connects event handlers, state, and component logic to the existing DOM.

Hydration mismatch

Akkor történik, ha a szerveren renderelt HTML és a kliensen első render során előálló React output nem egyezik. Tipikus ok: dátum/idő, random érték, browser-only API használata render közben.

Hooks definíció

Hook = React function, amivel function componentben React feature-öket használsz: state, effect, context, refs, memoization, külső subscription, custom reusable logic.

Angol válasz:

Hooks are functions that let React function components use React features such as state, side effects, context, refs, and memoization. They also make it possible to extract and reuse stateful logic between components through custom hooks.

- useState: local component state.
- useEffect: external systemekkel sync, subscription, cleanup, side effects.
- useContext: Context API érték olvasása.
- useMemo: drága számítás eredményének cache-elése két render között.
- useCallback: function reference cache-elése két render között.
- useRef: mutable érték vagy DOM reference render trigger nélkül.

5. Re-render optimalizálás és performance

Alap stratégia

Senior válasznál fontos: először mérni kell, nem reflexből memoizálni mindent. React performance optimalizálásnál a leggyakoribb gond a túl nagy parent state, változó reference-ű props, túl nagy listák, és unnecessary context updates.

Angol válasz:

I would first measure the problem before optimizing. Common techniques are keeping state as local as possible, avoiding unnecessary props, splitting large components, using React.memo for stable child components, memoizing expensive calculations with useMemo, memoizing callbacks with useCallback, and using memoized selectors for Redux state.

React.memo, useMemo, useCallback

- React.memo: component szinten mondd Reactnak, hogy ha a props nem változott, a child újrafuttatása kihagyható.
- useMemo: egy számított értéket cache-el. Hasznos drága számításnál vagy stabil object/array propnál.
- useCallback: function reference stabilizálására jó, főleg memoizált childoknál vagy effect dependencyknél.
- Nem kell mindenhova. Ha túlhasználsz, bonyolítja a kódot és nem biztos, hogy gyorsít.

```
const Child = React.memo(function Child({ title, onSave }) {  
  return <button onClick={onSave}>{title}</button>;  
});
```

```
function Parent() {  
  const [count, setCount] = useState(0);  
  const onSave = useCallback(() => save(), []);  
  return <Child title="Save" onSave={onSave} />;  
}
```

Felesleges props és reference változás

Amit te is emlékszel: ha felesleges paramétereket adsz át childnak, vagy minden rendernél új object/function referenciát hozol létre, akkor memoizált child is újra renderelhet. Ezért optimalizálásnál gyakori lépés a props API tisztítása.

Angol válasz:

When optimizing child re-renders, I try to avoid passing unnecessary props and avoid creating new object, array, or function references on every render unless needed. If a child is memoized with React.memo, stable props are important; otherwise the memoization cannot help much.

Nagy lista: 1 millió elem

Angol válasz:

I would not render one million elements directly. The first decision is usually product and UX related: do users really need all items at once? Technically I would use backend pagination or cursor-based loading, and on the frontend I would use virtualization/windowing so only the visible rows are mounted in the DOM. For example, react-window or react-virtualized can render a small visible window while the user scrolls through a large dataset.

- Pagination: egyszerre csak egy oldalnyi adat.
- Infinite scroll: fokozatos betöltés.
- Virtualization/windowing: csak a látható sorok vannak DOM-ban.
- Stable keys: React tudja követni az itemeket.
- Memoized row components: csak ahol tényleg mérhető előnye van.

6. State management: Redux nélkül és Reduxszal

State típusok

A jó válasz mindig azzal kezdődik, hogy nem egyetlen state-megoldás van. Más kell local UI state-re, server state-re, navigation state-re és complex shared app state-re.

State típusa	Jó megoldás	Példa
--------------	-------------	-------

Local UI state	useState / useReducer	modal open, selected tab
Simple shared state	Context API	theme, language, current user
Complex app state	Redux	cross-module state, app events
Server state	React Query / SWR	API data, cache, refetch
Navigation state	URL / router	filters, pagination, selected page

Props drilling

State-et és setttert végigadni propsként teljesen valid, ha 1-2 szintről van szó. Ha sok unrelated componenten kell keresztülvinni, prop drilling lesz belőle.

Angol válasz:

Passing state and setters through props is fine for small component trees or direct parent-child relationships. But if the same state has to pass through many unrelated intermediate components, it becomes prop drilling. At that point I would consider Context API, Context with useReducer, or Redux depending on complexity.

Senior kiegészítés

Nagy appban sokszor jobb domain-specific actionöket adni, nem raw setttereket: login(), logout(), changeLanguage(), updateFilter(). Így kontrolláltabb a state update.

Context API mint Redux alternatíva

Context API a React beépített megoldása arra, hogy értékeket elérhetővé tegyél egy component subtree számára anélkül, hogy minden szinten propként adnád tovább. Redux-szerűbbé useReducerrel válik.

Angol válasz:

By Context I mean the React Context API. It lets us provide values to a component subtree without passing them through every intermediate component as props. Combined with useReducer, it can be used as a lightweight alternative to Redux for simpler shared state. Context provides access, and useReducer provides predictable state updates.

```
const StateContext = createContext(null);
const DispatchContext = createContext(null);
```

```
function reducer(state, action) {
  switch (action.type) {
```

```

case "SET_LANGUAGE":
return { ...state, language: action.payload };
default:
return state;
}
}

function AppProvider({ children }) {
const [state, dispatch] = useReducer(reducer, initialState);
return (
<StateContext.Provider value={state}>
<DispatchContext.Provider value={dispatch}>
{children}
</DispatchContext.Provider>
</StateContext.Provider>
);
}

```

Mikor nem elég?

Context nem ad Redux DevTools-szerű debugingot, middleware modell, selector réteget vagy komplex async orchestrationt. Egy túl nagy context value sok consumer re-renderjét triggerelheti. Ezért érdemes domain szerint bontani: AuthContext, LanguageContext, FeatureFlagContext.

Redux: mikor indokolt?

Angol válasz:

Redux becomes useful when the application has complex shared state, multiple independent modules that need to react to the same application-level events, and a need for predictable state updates and debugging. I would not use it by default for every app, but for a large modular application it can make the behavior easier to reason about.

Selectors és reducers

- Reducer: pure function, amely az előző state + action alapján új state-et ad vissza.
- Selector: elrejtí a store shape-et a componentektől, derived data-t számol, memoizálható.
- Jó selector réteg mellett a UI nem függ közvetlenül a store belső struktúrájától.

Angol válasz:

Reducers define how state changes in response to actions. Selectors read

data from the store and can compute derived data. They also decouple components from the exact store shape, which makes refactoring easier.

Redux-Saga: miért?

A saga nem csak "API call library". Inkább async workflow és side-effect orchestration. Jó, ha több action, több API hívás, cancellation, retry, sequencing vagy cross-module coordination kell.

Angol válasz:

Redux-Saga was useful for more complex async workflows and side effects. We used it for API calls, sequencing, coordination between actions, error handling, and flows where multiple modules had to react to the same application-level events. Reducers stayed focused on state updates, while sagas handled the side effects and orchestration.

7. React vs Next.js, SSR, React 19

React vs Next.js

Ezt már kérdezték. A rövid különbség: React UI library; Next.js React framework production appokra. Next ad routingot, SSR/SSG-t, server components irányt, image/font/metadata optimalizációt, data fetching mintákat, file-based routingot.

Angol válasz:

React is a UI library for building components and user interfaces. Next.js is a framework built on top of React. It provides routing, server-side rendering, static generation, data fetching patterns, server/client component structure, image and metadata handling, and a more complete production application structure.

SPA mikor oké?

Internal dashboards, nagyon interaktív appok, auth mögötti alkalmazások vagy SEO szempontból kevésbé kritikus rendszerek esetén egy sima React SPA Vite-tal teljesen valid lehet.

SSR előnyei és hátrányai

- Előny: gyorsabb első vizuális megjelenés bizonyos esetekben.
- Előny: SEO és social preview jobb lehet, mert HTML már a szerveren előáll.
- Előny: gyengébb kliensen kevesebb initial rendering munka.
- Hátrány: komplexebb architecture, caching, server cost.
- Hátrány: hydration mismatch hibák lehetnek.
- Hátrány: nem minden component futtatható ugyanúgy szerveren és kliensen.

Angol válasz:

SSR means the server sends pre-rendered HTML to the browser. This can improve SEO, social previews, and perceived initial load time. The tradeoff is additional server-side complexity, caching concerns, and hydration-related issues.

React 19: amit elég tudni

Nem kell úgy tenni, mintha mély production tapasztalatod lenne React 19-cel. A korrekt állítás: főleg React 18-style production appokon dolgoztál, de ismered a React 19 fő irányait.

- Actions / async transitions: jobb pending, error, optimistic és form flow támogatás.
- useActionState, useFormStatus, useOptimistic: főleg form submit és optimistic UI környéke.
- ref propként: forwardRef bizonyos esetekben kevésbé szükséges.
- Document metadata: title/meta/link komponensekből is kezelhető modern környezetben.
- Server Components / SSR irány: frameworkökben fontosabb, például Next.js-nél.

Angol válasz:

My production experience is mostly with React 18-style applications. I have not used React 19 deeply in production yet, but I am aware of the main direction: better support for async actions and forms, optimistic updates, simpler ref handling, document metadata support, and improvements around server rendering and modern React frameworks.

8. App struktúra, reusable components, Figma

Route-based vs feature-based structure

Route alapján szervezni nem rossz, de nagy appban a route inkább belépési pont. A tényleges ownership és logika gyakran feature-határok mentén jobb.

Angol válasz:

For small applications, a route-based structure can be enough. For larger React applications, I usually prefer feature-based boundaries, with routes acting as entry points. A route or page composes feature modules, while the components, hooks, API calls, selectors, and state related to a feature live close to that feature. Truly reusable UI elements go into a shared layer.

```
src/  
app/ // providers, store, routing setup
```

```
pages/ // route/page entry points
features/ // user, payments, dashboard, permissions
shared/ // generic UI, utils, hooks, types
```

Reusable component design

Itt senior válasz kell: nem kell mindent túl korán általánosítani. Reusable component akkor jó, ha valós ismétlődő UI/behavior pattern van.

Angol válasz:

I try to create reusable components when the same UI pattern or behavior appears in multiple places, but I avoid over-generalizing too early. A good reusable component should have a clear API, predictable behavior, sensible defaults, and should not contain too much product-specific logic. If a component becomes too configurable, it may be better to split it into smaller pieces.

Figma designből fejlesztés

- Nézd a missing state-eket: loading, error, empty, disabled, permission denied.
- Responsive viselkedés: mobile/tablet/desktop, hosszú text, localization miatt hosszabb szövegek.
- Interakciók: hover, focus, keyboard navigation.
- Reusable pattern: van-e már meglévő component, vagy új pattern kell?
- Designerrel early clarification: ne a végén derüljön ki, hogy edge case nincs megtervezve.

Angol válasz:

When working from Figma designs, I check not only the happy path but also missing states: loading, empty, error, disabled, permissions, responsiveness, and long localized text. I also look for reusable patterns and discuss unclear edge cases early with design or product.

Accessibility basics

- Semantic HTML: button legyen button, link legyen link.
- Keyboard navigation és focus state.
- Label form inputokhoz.
- ARIA csak ahol kell, nem mindenre automatikusan.

- Kontraszt, focus trap modalban, screen reader szempontok.
- Error message kapcsolódjon a mezőhöz.

Angol válasz:

I usually start with semantic HTML, proper labels, keyboard navigation, visible focus states, and using ARIA only where it is needed. For forms and modals, focus management and clear error messages are especially important.

9. Legacy migration

jQuery/Rails/HAML/ERB to React

Ez a hirdetés egyik legerősebb pontja. Készülj arra, hogy nem csak modern React fejlesztés lesz, hanem meglévő template/jQuery részek fokozatos Reactesítése.

Angol válasz:

I would approach it incrementally. First I would understand the existing behavior and identify isolated areas where React can be introduced safely. I would avoid a full rewrite unless there is a strong reason. I would migrate in small reviewable steps, keep existing behavior stable, add tests around critical paths, and gradually move new functionality into React while reducing the legacy surface area.

Mit refaktorálj és mit hagyj békén?

Angol válasz:

I would prioritize areas that are actively changing, risky, hard to maintain, or blocking new product work. I would avoid refactoring stable legacy code just for style reasons. The migration should reduce risk and support product goals, not create a long rewrite project with unclear value.

Régi React migrálása modern Reactre

- Dependency audit: React, router, state library, build tool, test libs.
- Deprecated API-k: old lifecycle methods, legacy context, ReactDOM.render -> createRoot.
- Class componenteket nem kell automatikusan hooksra írni, csak ha aktívan módosulnak vagy gondot okoznak.
- Test critical flows before és after migration.
- Small steps: dependency update, build fix, tests, then pattern cleanup.

Angol válasz:

For an older React migration, I would first check dependencies, deprecated APIs, test coverage, and build tooling. I would update incrementally, replace deprecated patterns where needed, move from ReactDOM.render to createRoot for React 18+, and test critical flows. I would not rewrite all class components to hooks automatically unless

there is a concrete reason.

10. CSS és JavaScript fundamentals

CSS: mire készülhetsz

- Box model: content, padding, border, margin; box-sizing: border-box.
- Flexbox: egydimenziós layout, sor vagy oszlop, alignment.
- Grid: kétdimenziós layout, sorok és oszlopok.
- Position: static, relative, absolute, fixed, sticky.
- Specificity: melyik CSS rule nyer; inline > id > class > element.
- Responsive design: media queries, fluid layout, min/max width, wrapping.
- CSS Modules / CSS-in-JS: scope-olt styling, naming collision csökkentés.

Angol válasz, ha általánosan kérdezik:

I am comfortable with practical CSS for application UIs: layout with flexbox and grid, responsive behavior, component-level styling, CSS specificity, and accessibility-related details such as focus states. I have worked more in product UI contexts than in highly artistic landing-page CSS.

JavaScript: mire készülhetsz

- Closure: function hozzáfér a külső lexical scope változóihoz.
- Promise / async-await: aszinkron műveletek kezelése.
- Event loop: call stack, microtasks, macrotasks, async callbackok sorrendje.
- Value vs reference: primitive értékek vs object/array referenciák.
- Immutability: React/Redux state-et nem módosítunk in-place.
- Debounce/throttle: gyakori eventek kontrollálása, pl. search input/scroll.
- ES modules: import/export, code splitting, bundling.

Angol válasz, ha általánosan kérdezik:

On the JavaScript side, the most relevant areas for frontend work are async programming with promises and async/await, closures, event handling, object references and immutability, modules, and performance patterns like debounce or throttle.

11. Code quality, testing, release safety

Code quality toolok

Ezt már kérdezték: legyen kész listád és tudd rétegekben elmondani.

- TypeScript typecheck: type hibák build/runtime előtt.
- ESLint: static analysis, bad patterns, hooks szabályok, unused vars.
- Prettier: egységes formázás, kevesebb style vita PR-ban.
- Unit/component tests: Jest/Vitest + React Testing Library.
- E2E tests: Playwright/Cypress/Selenium kritikus user flowkra.
- Visual regression: Loki/Chromatic-style screenshot comparison.
- CI checks: typecheck, lint, tests, build, audit merge előtt.
- Code review: design, maintainability, edge cases, test coverage.
- Dependency audit: npm audit, Snyk, Dependabot jellegű megoldások.

Angol válasz:

For frontend quality, I usually think in layers: TypeScript type checking, ESLint, Prettier, unit and component tests, visual regression tests, E2E tests for critical flows, and CI checks before merge. Tooling helps, but code review and clear ownership are still important.

Regression testing

Regression test azt ellenőrzi, hogy egy új változtatás nem rontott-e el korábban működő funkcionalitást. A visual regression ennek UI kinézetre fókuszáló verziója.

Angol válasz:

Regression testing means checking that new changes did not break existing functionality. Visual regression testing applies the same idea to the UI: tools like Loki compare screenshots to detect unintended visual changes in components or pages.

Release safety és feature flags

- Feature flag: feature kapcsolható deploy nélkül.
- Gradual rollout: kisebb kockázat, először belső user/QA/szűk audience.
- Kill switch: problémás feature kikapcsolható rollback nélkül.
- A/B vagy product-specific config: feature más termékben másképp működhet.
- Veszély: flag debt, túl sok régi flag marad a kódban.

Angol válasz:

Feature flags helped us reduce release risk. We could deploy code separately from enabling the feature, test it with a limited audience, turn it off quickly if needed, and support product-specific configurations. The main thing to manage is flag cleanup, otherwise old flags become technical debt.

12. REST, GraphQL, API collaboration

REST jelentése és lényege

REST = Representational State Transfer. Interjún nem elég a rövidítést tudni; mondd el, hogy resource-alapú API architectural style HTTP methodokkal.

Angol válasz:

REST stands for Representational State Transfer. It is an architectural style for APIs where resources are identified by URLs, and HTTP methods define the operation: GET for reading, POST for creating, PUT or PATCH for updating, and DELETE for removing resources.

```
GET /users // list users
GET /users/123 // get one user
POST /users // create user
PATCH /users/123 // partial update
DELETE /users/123 // delete user
```

GraphQL: miért lehet hasznos?

Nálad jó sztori: payload csökkentés, frontend data shape kontroll, backend schema együttműködés. Ne állítsd, hogy mindig jobb, mint REST.

Angol válasz:

GraphQL can be useful when the frontend needs more control over the shape of the data, or when REST endpoints would return too much or too little data. In our case it helped reduce payload size in targeted flows and made some data fetching more explicit. But REST is still simpler and perfectly fine for many resource-based operations.

API contract collaboration

- Request/response shape egyeztetés.
- Error codes és error body format.
- Loading/empty/error states a UI-ban.
- Pagination: offset vs cursor.
- Permissions/auth: 401/403 kezelése.
- Backward compatibility migrationnél.

Angol válasz:

When working with backend teams, I try to clarify the API contract early: request and response shapes, validation errors, permissions, pagination, loading and empty states, and backward compatibility. This avoids frontend assumptions and makes the UI behavior more predictable.

13. AI-assisted development

Mivel náluk elhangzott, hogy Figma -> UI generálás és AI-assisted development erős, erre pozitívan, de nem kritikátlanul válaszolj. A legjobb pozíció: AI-t gyorsítónak használd, de a felelősség az engineeré.

Angol válasz:

I use AI as a development accelerator, especially for boilerplate, refactoring suggestions, test generation, documentation, and exploring implementation options. But I would not treat generated code as automatically correct. I still review architecture, edge cases, types, accessibility, performance, security, and maintainability. For me, AI is useful when it speeds up implementation, while the engineer remains responsible for quality and correctness.

Ha azt mondják: “mi már nem is nagyon írunk kódot”

Válasz irány:

That sounds interesting, and I would be curious to learn how you organize the workflow around generated code. I think the important part is having strong review, tests, type checking, and architectural ownership, because AI can speed up implementation but does not remove the need for engineering judgment.

14. További gyakorló kérdések mintaválasszal

Q: What is the difference between controlled and uncontrolled components?

Interjú válasz:

In a controlled component, React state is the source of truth for the input value. In an uncontrolled component, the DOM keeps its own state and React reads it through refs when needed. Controlled components are common when validation and UI state depend on the value, but uncontrolled approaches can be useful for simpler forms or performance-sensitive cases.

Mit figyelnek: Formoknál klasszikus kérdés.

Q: What should go into Redux and what should stay local?

Interjú válasz:

Local UI state, such as an open dropdown or selected tab, should usually stay local. Redux is better for shared application state used by multiple unrelated parts of the UI, cross-module events, session-level state, or complex workflows where predictable updates and debugging matter.

Mit figyelnek: Local vs shared app state.

Q: When would you use React Query?

Interjú válasz:

I would use React Query for server state: API data that needs caching, loading states, retries, refetching, invalidation, and stale data handling. It is often better than manually storing API responses in Redux, while Redux remains useful for complex client/application state.

Mit figyelnek: Modern senior válasz.

Q: How do you handle loading, empty and error states?

Interjú válasz:

I try to make them explicit states in the UI. Loading should communicate

progress, empty state should explain why there is no data and what the user can do next, and error state should be recoverable where possible, for example with retry or clear guidance.

Mit figyelnek: UI maturity.

Q: How do you work with code you did not write?

Interjú válasz:

First I try to understand the existing behavior and constraints before changing it. I read surrounding code, check tests and usage, make small changes, and avoid large refactors unless there is a clear reason. With legacy code, safety and behavior preservation are more important than style preferences.

Mit figyelnek: Legacy role miatt fontos.

Q: How do you mentor less experienced developers?

Interjú válasz:

I try to help with context, not just corrections. In code review I explain the reason behind suggestions, point to patterns used in the codebase, and pair when the topic is complex. The goal is to improve the code and also help the developer become more independent next time.

Mit figyelnek: Hirdetésben mentoring szerepel.

Q: What is your approach to code review?

Interjú válasz:

I look for correctness, readability, edge cases, type safety, test coverage, accessibility, and whether the solution fits existing patterns. I try to keep comments constructive and separate must-fix issues from optional suggestions.

Mit figyelnek: Senior collaboration.

Q: How do you decide if a component is too generic?

Interjú válasz:

If the component API becomes full of flags and special cases, it may be too generic. I prefer components that solve a clear pattern well.

Sometimes composition or smaller components are better than one large highly configurable component.

Mit figyelnek: Reusable components maturity.

Q: How did localization affect UI development?

Interjú válasz:

Localization affects layout because translated text can be longer, shorter, or have different formatting needs. I try to avoid hardcoded text, consider dynamic lengths, and make sure components handle longer labels, empty values, and language-specific formatting.

Mit figyelnek: Hirdetésben i18n plusz.

Q: What is the advantage of TypeScript in frontend apps?

Interjú válasz:

TypeScript helps catch errors earlier, documents data shapes, improves refactoring safety, and makes API contracts and component props clearer. It does not replace tests, but it reduces many classes of runtime errors and improves maintainability in larger codebases.

Mit figyelnek: Tooling/code quality.

Q: How would you debug a production frontend issue?

Interjú válasz:

I would start from reproduction steps and logs, then check monitoring tools such as Sentry or Kibana, browser console/network data, feature flag state, recent releases, and API responses. If needed, I would disable a risky feature flag or prepare a small rollback/fix.

Mit figyelnek: Nálad Sentry/Kibana tapasztalat jó plusz.

Q: How do you avoid breaking behavior during refactor?

Interjú válasz:

I keep changes small, add or rely on tests around the behavior, compare before and after behavior, use feature flags if the change is risky, and ask for review from people who know the area. Refactoring should be behavior-preserving unless the product explicitly changes behavior.

Mit figyelnek: Migration/refactor core.

Q: What are common causes of hydration mismatch?

Interjú válasz:

Common causes are rendering time-dependent or random values, using browser-only APIs during server render, different locale formatting between server and client, or conditional rendering that differs between server and browser. The fix is usually to make the first render deterministic or move client-only logic into effects or client components.

Mit figyelnek: Next/SSR kérdéshez.

Q: How do you handle API errors in the UI?

Interjú válasz:

I distinguish between validation errors, permission errors, network/server errors, and empty states. The UI should show user-friendly messages, preserve user input where possible, and provide retry or recovery options when appropriate.

Mit figyelnek: API collaboration.

Q: What are the tradeoffs of AI-generated code?

Interjú válasz:

AI can speed up boilerplate, tests, refactors, and exploration, but generated code can still be wrong, overcomplicated, insecure, or inconsistent with the architecture. I would treat it like code from a junior developer: useful, but it needs review, tests, and ownership.

Mit figyelnek: AI-heavy cégnél jó válasz.

15. Utolsó napos fókuszlista

- Gyakorold a 30 másodperces és 2 perces bemutatkozást hangosan.
- Tudd elmondani a modular React appot: core, modules, events/state, extension points, Redux/Saga.
- React fundamentals: re-render triggers, virtual DOM, reconciliation, hydration.

- State management: props drilling, Context API + useReducer, Redux, Saga, React Query.
- Performance: React.memo, stable props, useMemo/useCallback, large list virtualization.
- Migration: jQuery/Rails vagy régi React fokozatos modernizálása.
- Tooling: TypeScript, ESLint, Prettier, tests, visual regression, CI.
- AI-assisted development: positive, de quality ownership hangsúlyal.
- Készülj 2 konkrét sztorival: egy technical challenge és egy release/coordination/automation példa.

16. Rövid referenciaanyag

A React/Next.js részeknél a hivatalos dokumentáció és blogok alapján készült rövid összefoglalók:

- React 19 announcement: <https://react.dev/blog/2024/12/05/react-19>
- React hydrateRoot: <https://react.dev/reference/react-dom/client/hydrateRoot>
- React createRoot: <https://react.dev/reference/react-dom/client/createRoot>
- React memo: <https://react.dev/reference/react/memo>
- React useMemo: <https://react.dev/reference/react/useMemo>
- React useCallback: <https://react.dev/reference/react/useCallback>
- React useContext: <https://react.dev/reference/react/useContext>
- Next.js Server and Client Components: <https://nextjs.org/docs/app/getting-started/server-and-client-components>
- Next.js hydration error explanation: <https://nextjs.org/docs/messages/react-hydration-error>